

Elaborazione e aggregazione di dati privacy-aware con tecniche omomorfiche

Doru Ionut Parvu

Indice

1	Applicazioni di elaborazioni omomorfe a scenari medicali	3
1.1	Introduzione	3
1.2	Richiami sulla regressione lineare	4
1.2.1	Funzione di costo	4
1.2.2	Determinazione dei coefficienti	5
1.2.3	Interpretazione in ambito medico	5
1.3	Regressione lineare ed elaborazione omomorfa	6
1.4	Scenario 1: query cifrata al computation server	7
1.4.1	Schema logico	7
1.4.2	Distribuzione delle chiavi	8
1.5	Scenario 2: il polinomio cifrato viene trasferito al decryptor	9
1.5.1	Schema logico	9
1.5.2	Distribuzione delle chiavi	10
1.6	Confronto tra le due architetture	11
1.7	Esempio applicativo: stima dei giorni di ricovero	12
1.7.1	Come parte il training	12
1.7.2	Quale modello viene recuperato	12
1.7.3	Verifica su un nuovo dato	12
1.7.4	Lettura dell'esempio nei due scenari	13
1.8	Aspetti operativi e limiti	13
1.9	Conclusioni	13
2	Protocollo di aggregazione simil prio3 utilizzando tecniche omomorfe	14
2.1	Introduzione	14
2.2	Idea generale di fhe-vdaf-0	14
2.3	Ruoli architetturali e gestione delle chiavi	15
2.3.1	Attori del sistema	15
2.3.2	Chi possiede quali chiavi	16
2.3.3	Conseguenze di sicurezza	16
2.4	Architettura complessiva in uno scenario di dati aggregati	16
2.4.1	Schema dei messaggi	17
2.5	Conversione dell'input in binario	17
2.5.1	Perché usare una rappresentazione binaria	17
2.5.2	Vincolo sul valore massimo	17
2.5.3	Esempi di codifica	18

2.6	Validazione omomorfica dell'input	19
2.6.1	Controllo che ogni slot sia un bit	19
2.6.2	Da errore a flag di validità	20
2.6.3	Validate-or-zero	20
2.7	Aggregazione dei report validi	21
2.8	Esempio completo su dati aggregati	21
2.8.1	Codifica dei valori	21
2.8.2	Somma per slot	21
2.8.3	Decodifica finale	21
2.9	Punti di forza e limiti architetturali	22
2.9.1	Vantaggi	22
2.9.2	Limiti	22
2.10	Conclusioni	22

Capitolo 1

Applicazioni di elaborazioni omomorfiche a scenari medicali

1.1 Introduzione

In questo capitolo si considera un'applicazione concreta delle tecniche descritte in precedenza: l'uso della regressione lineare in uno scenario medico. L'idea di fondo è semplice. Più strutture sanitarie o più soggetti che raccolgono dati clinici vogliono contribuire alla costruzione di un modello statistico, ma senza rinunciare alla protezione delle informazioni trattate.

Nel caso concreto considerato, la regressione lineare viene usata per stimare il numero di giorni di permanenza in ospedale di un paziente a partire da età, indice di comorbidità di Charlson, numero di procedure precedenti, pressione sistolica e diastolica e indice di massa corporea. Il capitolo mostra come questo modello venga inserito in un'architettura i cui attori principali sono:

- il *setup authority*, che crea il contesto crittografico e genera le chiavi del sistema;
- il *data provider*, che possiede i dati clinici originari;
- il *computation server*, che esegue il training del modello o le valutazioni numeriche;
- il *query actor*, che desidera interrogare il modello;
- il *decryptor*, che può coincidere con un soggetto istituzionale, ad esempio il SSN, e che detiene la capacità di decifrare i risultati.

La separazione tra computation server e decryptor è intenzionale. Le computazioni omomorfiche possono richiedere tempi elevati e possono quindi essere affidate anche a soggetti terzi specializzati, senza esporre in chiaro i dati sensibili o le query dei pazienti. Il decryptor resta invece il soggetto che possiede la chiave segreta e rende leggibile solo il risultato finale. A monte, il setup authority inizializza il sistema e distribuisce chiave pubblica, chiave segreta ed eventuali evaluation keys agli attori autorizzati.

Nel primo scenario il query actor invia query cifrate al computation server e riceve un risultato cifrato, poi decifrato dal decryptor. Nel secondo il computation server invia il polinomio cifrato al decryptor, mentre il query actor interroga in chiaro il servizio del soggetto fiduciario.

1.2 Richiami sulla regressione lineare

La regressione lineare serve a trovare una relazione semplice tra dati in ingresso e dato in uscita. In altre parole, si parte da esempi già osservati e si cerca una formula che descriva il loro andamento medio.

Il dataset contiene più osservazioni, una per ciascun paziente. Ogni osservazione comprende sei valori di ingresso: età, indice di comorbilità di Charlson, numero di procedure precedenti, pressione sistolica, pressione diastolica e indice di massa corporea. A questi sei valori corrisponde un unico valore da prevedere, cioè il numero di giorni di permanenza in ospedale.

Nel caso con più variabili in ingresso, il modello si scrive come

$$\hat{y} = w^T x + b = \sum_{j=1}^6 w_j x_j + b, \quad (1.1)$$

dove:

- w contiene i coefficienti del modello;
- x contiene i sei valori di ingresso relativi al paziente;
- b è il bias;
- $\hat{y}$ è il numero stimato di giorni di permanenza in ospedale.

La regressione adottata è lineare perché tutte le variabili x_j compaiono esclusivamente alla prima potenza: il modello non contiene termini quadratici, potenze superiori o interazioni tra feature. Questa scelta costituisce una semplificazione intenzionale. Variabili come la pressione sistolica e diastolica possono infatti avere una relazione più complessa e non lineare con la durata del ricovero; in un'applicazione clinica completa si potrebbero quindi considerare trasformazioni delle feature o modelli differenti. Nel presente caso si mantiene invece una funzione affine di primo grado, poiché lo scopo principale è dimostrare in modo semplice e leggibile l'applicazione delle tecniche omomorfiche.

1.2.1 Funzione di costo

Per capire se la retta trovata è buona oppure no, bisogna confrontare le previsioni del modello con i valori reali. Durante l'ottimizzazione si considera la metà dell'errore quadratico medio:

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}_i - y_i)^2. \quad (1.2)$$

Il fattore $1/2$ semplifica il calcolo del gradiente e non cambia i valori di w e b che minimizzano la funzione. La MSE completa, pari a $2J(w, b)$, viene usata per monitorare l'andamento del training. Se $J(w, b)$ è piccolo, il modello rappresenta bene le osservazioni disponibili.

1.2.2 Determinazione dei coefficienti

I coefficienti non vengono ricavati calcolando direttamente l'inversa di una matrice. Il modello usa invece la *discesa del gradiente* (*gradient descent*), un procedimento iterativo che parte da coefficienti iniziali molto semplici e li corregge poco alla volta nella direzione che riduce la funzione di costo.

Il training parte da pesi e bias nulli e, per un numero prefissato di epoche, calcola la previsione, misura l'errore e aggiorna i coefficienti usando un learning rate η .

In forma compatta, la previsione del modello è

$$\hat{y} = Xw + b\mathbf{1}_n, \quad (1.3)$$

dove $\mathbf{1}_n$ è un vettore di n elementi uguali a uno e indica che lo stesso bias viene aggiunto alla previsione di ogni paziente. Successivamente si calcola il vettore degli errori:

$$E = \hat{y} - y. \quad (1.4)$$

Se una previsione è troppo alta rispetto al valore reale, l'errore sarà positivo; se invece è troppo bassa, l'errore sarà negativo.

Una volta ottenuto l'errore, il modello aggiorna i pesi tramite

$$w \leftarrow w - \eta \frac{1}{n} X^T E \quad (1.5)$$

e aggiorna il bias tramite

$$b \leftarrow b - \eta \cdot \frac{1}{n} \sum_{i=1}^n E_i. \quad (1.6)$$

Se il modello sbaglia sistematicamente in una direzione, i coefficienti vengono spostati gradualmente nella direzione opposta. La MSE viene calcolata a ogni epoca per verificare il miglioramento.

1.2.3 Interpretazione in ambito medico

Nel caso considerato, il modello assume la forma

$$\hat{y} = w_1 \text{age} + w_2 \text{cci} + w_3 \text{num_procedures} + w_4 \text{systolic} + w_5 \text{diastolic} + w_6 \text{bmi} + b, \quad (1.7)$$

dove $\hat{y}$ è la permanenza stimata in ospedale, espressa in giorni. I coefficienti descrivono il contributo delle singole feature:

- w_1 è associato all'età del paziente, compresa nell'intervallo $[0, 100]$ anni;
- w_2 è associato al Charlson Comorbidity Index, compreso nell'intervallo $[0, 37]$;

- w_3 è associato al numero di procedure o interventi precedenti, compreso nell'intervallo $[0, 20]$;
- w_4 è associato alla pressione sistolica, compresa nell'intervallo $[80, 200]$ mmHg;
- w_5 è associato alla pressione diastolica, compresa nell'intervallo $[50, 120]$ mmHg;
- w_6 è associato all'indice di massa corporea, compreso indicativamente nell'intervallo $[15, 40]$;
- b rappresenta il bias del modello.

Questi intervalli descrivono i valori attesi delle variabili di ingresso, non i possibili valori dei coefficienti appresi. Inoltre, i coefficienti esprimono associazioni apprese dal dataset e non relazioni causali.

1.3 Regressione lineare ed elaborazione omomorfica

Una volta costruito il modello, il passaggio successivo consiste nel capire come usarlo nei due scenari considerati. La regressione lineare si presta bene a questo contesto perché, una volta noti i coefficienti, la previsione si ottiene con un calcolo molto semplice:

$$\hat{y} = w^T x + b = \sum_{j=1}^d w_j x_j + b. \quad (1.8)$$

In sostanza, bisogna fare prodotti e somme. Questo rende il modello adatto a un uso pratico anche in un'architettura protetta, perché la fase di inferenza non richiede logiche troppo complicate.

L'implementazione in FHE usa lo schema CKKS. Questa scelta è utile perché consente di rappresentare ed elaborare anche valori non interi, caratteristica importante in regressione lineare quando coefficienti, medie o risultati intermedi non sono interi. Inoltre, nell'implementazione si ricorre al *bootstrapping* per riazzerare la profondità moltiplicativa del ciphertext, poiché durante la regressione lineare, in particolare nelle fasi iterative di calcolo, si accumulano molte moltiplicazioni omomorfe. CKKS introduce approssimazioni numeriche e quindi piccoli errori di arrotondamento, ma nel contesto considerato tali differenze sono trascurabili ai fini del risultato finale.

Nel calcolo in chiaro si può pensare a x , w ed y come a vettori e matrici ordinarie. In FHE, invece, questi dati devono essere inseriti negli slot di un ciphertext. Per questo motivo l'implementazione non lavora direttamente con la matrice nella sua forma più intuitiva, ma la riorganizza in blocchi che possano essere sommati e ruotati in modo efficiente.

Nel presente contesto si può scomporre il problema in due fasi:

- training del modello:** il computation server riceve i dati dai provider e costruisce i coefficienti della regressione;

- (b) **inferenza o query**: un attore interroga il modello per ottenere una previsione su un nuovo input.

Il punto interessante è che nei due scenari cambia soprattutto la fase di query. Il modello matematico resta lo stesso, ma cambia chi vede i dati in chiaro, chi esegue la valutazione finale e chi possiede le chiavi necessarie per decifrare.

1.4 Scenario 1: query cifrata al computation server

Nel primo scenario, il data provider invia i dati al computation server, che esegue la regressione lineare e costruisce il polinomio del modello. Successivamente, il query actor non invia un input in chiaro, ma una query cifrata. Il computation server valuta il modello sul dato cifrato e restituisce un risultato anch'esso cifrato. Infine, il query actor si rivolge a un servizio di decrypt, che può essere incarnato dal SSN, per ottenere il valore in chiaro.

1.4.1 Schema logico

Lo schema può essere rappresentato come segue, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Data Provider}_i \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(\text{record di training}_i), \quad (1.9)$$

$$\text{Computation Server} : \text{training su dati cifrati e costruzione di } \text{Enc}_{pk}(w, b), \quad (1.10)$$

$$\text{Query Actor} \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(x_q), \quad (1.11)$$

$$\text{Computation Server} : \text{valutazione del modello su } \text{Enc}_{pk}(x_q), \quad (1.12)$$

$$\text{Computation Server} \longrightarrow \text{Query Actor} : \text{Enc}_{pk}(w^T x_q + b), \quad (1.13)$$

$$\text{Query Actor} \longrightarrow \text{Decryptor/SSN} : \text{Enc}_{pk}(w^T x_q + b), \quad (1.14)$$

$$\text{Decryptor/SSN} : \text{Dec}_{sk}(\text{Enc}_{pk}(w^T x_q + b)), \quad (1.15)$$

$$\text{Decryptor/SSN} \longrightarrow \text{Query Actor} : w^T x_q + b. \quad (1.16)$$

Il computation server addestra il modello e valuta il polinomio su query cifrate; il query actor vede il risultato in chiaro solo dopo l'intervento del decryptor.

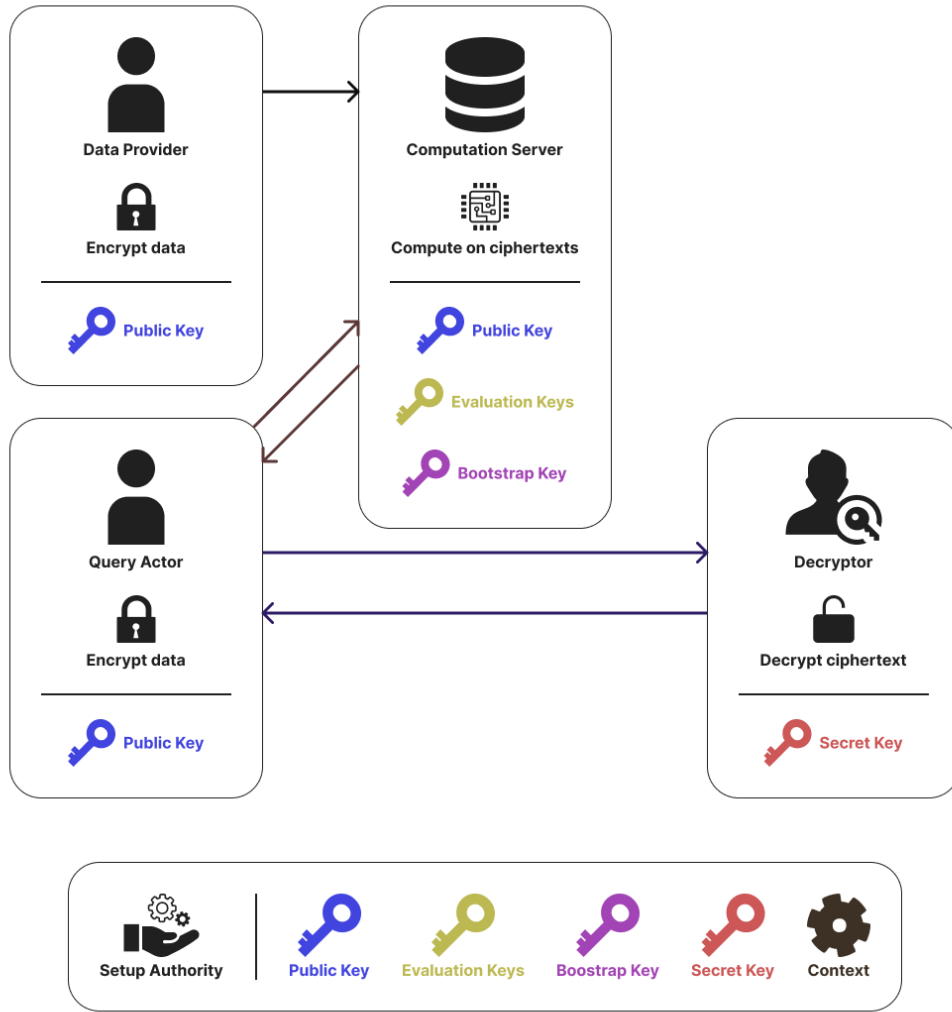


Figura 1.1: Schema dello scenario 1.

1.4.2 Distribuzione delle chiavi

In questo scenario si assume tipicamente la seguente ripartizione:

- il **setup authority** genera il contesto crittografico, la chiave pubblica pk , la chiave segreta sk ed eventuali evaluation keys, poi distribuisce tale materiale ai ruoli autorizzati;
- i **data provider** inviano al computation server soltanto dati cifrati con pk ;
- il **computation server** conosce la chiave pubblica pk e le eventuali evaluation keys necessarie alle operazioni omomorfiche;
- il **query actor** possiede o riceve la chiave pubblica pk per cifrare le query;
- il **decryptor/SSN** possiede la chiave segreta sk ;

La proprietà essenziale è che il computation server non dispone di sk , dunque non può vedere il contenuto delle query finali né dei risultati cifrati che produce.

Lo scenario protegge la query e separa chi calcola da chi decifra. Richiede però cifratura lato utente, gestione delle evaluation keys e l'intervento del decryptor, con maggiore complessità e latenza.

1.5 Scenario 2: il polinomio cifrato viene trasferito al decryptor

Nel secondo scenario, il data provider invia i dati al computation server, che esegue il training del modello. Una volta ottenuti i coefficienti della regressione, il computation server cifra il polinomio e lo trasferisce direttamente al decryptor, ad esempio il SSN. Da quel momento, il query actor non interagisce più con il computation server per la fase di inferenza, ma invia le proprie query in chiaro al soggetto fiduciario.

1.5.1 Schema logico

Lo schema può essere espresso come segue, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Data Provider}_i \longrightarrow \text{Computation Server} : \text{Enc}_{pk}(\text{record di training}_i), \quad (1.17)$$

$$\text{Computation Server} : \text{training su dati cifrati e costruzione di } \text{Enc}_{pk}(w, b), \quad (1.18)$$

$$\text{Computation Server} \longrightarrow \text{Decryptor/SSN} : \text{Enc}_{pk}(w, b), \quad (1.19)$$

$$\text{Decryptor/SSN} : \text{Dec}_{sk}(\text{Enc}_{pk}(w, b)), \quad (1.20)$$

$$\text{Query Actor} \longrightarrow \text{Decryptor/SSN} : x_q \text{ in chiaro}, \quad (1.21)$$

$$\text{Decryptor/SSN} \longrightarrow \text{Query Actor} : w^T x_q + b. \quad (1.22)$$

Il query actor beneficia di un'interfaccia più semplice: non cifra nulla, ma si appoggia a un'entità trusted che custodisce il modello e produce le risposte.

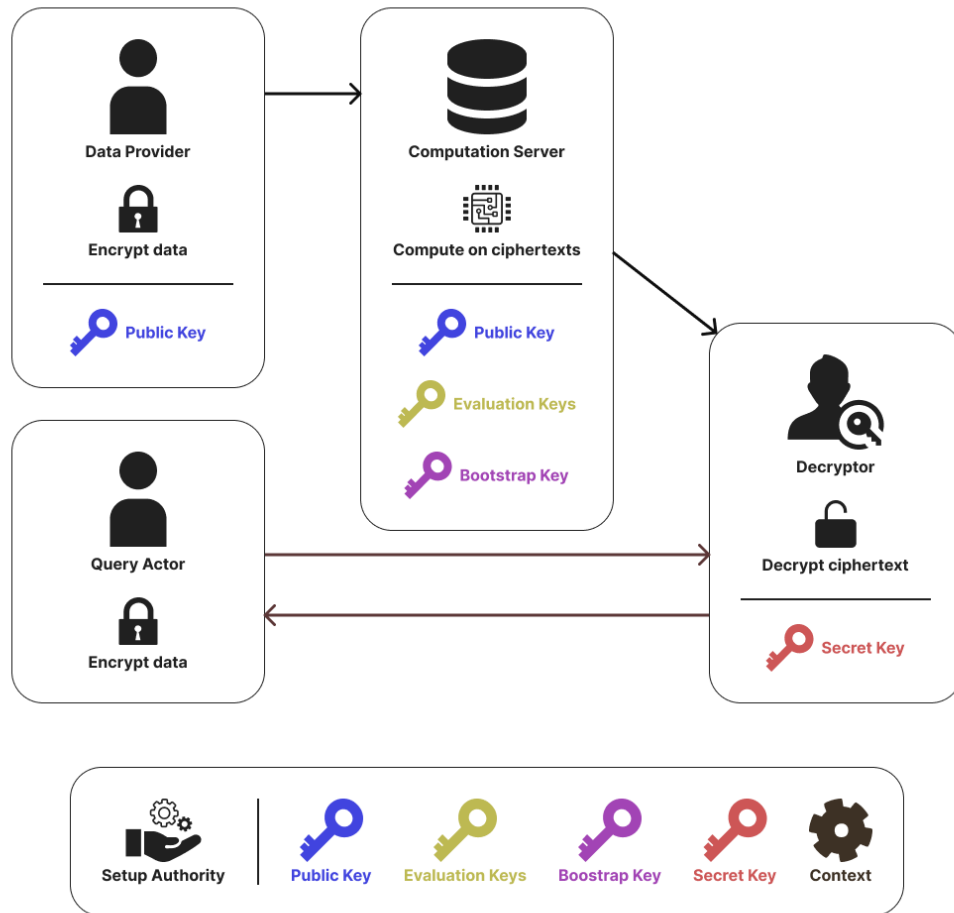


Figura 1.2: Schema dello scenario 2.

1.5.2 Distribuzione delle chiavi

La distribuzione delle chiavi è più centralizzata:

- il **setup authority** genera il contesto crittografico, la chiave pubblica pk , la chiave segreta sk ed eventuali evaluation keys, poi le distribuisce agli attori autorizzati;
- i **data provider** inviano al computation server soltanto dati cifrati con pk ;
- il **computation server** conosce la chiave pubblica pk e le eventuali evaluation keys necessarie alle operazioni omomorfiche;
- il **query actor** non ha bisogno di utilizzare direttamente chiavi crittografiche nella fase di interrogazione;
- il **decryptor/SSN** possiede la chiave segreta sk ;

Questa architettura è coerente con contesti in cui il decryptor rappresenta un ente istituzionale già riconosciuto come fiduciario, in grado di custodire chiavi, modelli e risultati.

Lo scenario semplifica l'interrogazione e riduce la latenza, ma espone l'input al decryptor e concentra informazioni e fiducia presso tale ente. È quindi adatto quando la fiducia nel soggetto istituzionale è alta.

1.6 Confronto tra le due architetture

Le due architetture risolvono lo stesso problema, ma ottimizzano aspetti diversi. Una sintesi comparativa può essere espressa nella Tabella 1.1.

Aspetto	Scenario 1	Scenario 2
Protezione delle query	Alta: il computation server riceve query cifrate	Più bassa: il query actor invia la query in chiaro al decryptor/SSN
Ruolo del computation server	Addestra il modello e valuta query cifrate	Addestra il modello e invia il polinomio cifrato al decryptor
Ruolo del decryptor	Decifra il risultato finale	Custodisce il modello e risponde alle query
Chiave segreta	Presso il decryptor/SSN	Presso il decryptor/SSN
Chiave pubblica	Usata da query actor e computation server	Usata da query actor e computation server
Complessità per l'utente finale	Maggiore: deve cifrare la query	Minore: usa un'interfaccia in chiaro
Latenza complessiva	Più elevata	Più contenuta, ma con costo di inferenza a carico del decryptor/SSN

Tabella 1.1: Confronto tra i due scenari di impiego della regressione lineare in ambito omomorfico-medicale.

Da questa tabella emerge un trade-off classico. Se si vuole proteggere fortemente la riservatezza dell'input di inferenza, lo scenario 1 è generalmente preferibile. Se invece si desidera un servizio più semplice, controllato da un ente sanitario centrale e più immediato da usare, lo scenario 2 può risultare più naturale. Va però ricordato che, nello scenario 2, il training resta a carico del computation server mentre l'inferenza viene eseguita dal decryptor, cioè dal SSN: anche questa fase ha quindi un costo computazionale e operativo che deve essere sostenuto dall'attore ultimo invece del server computazionale che però essendo in chiaro risultat molto meno dispendioso del eseguire una decifrazione.

1.7 Esempio applicativo: stima dei giorni di ricovero

Il dataset sanitario contiene, per ogni osservazione, le sei caratteristiche del paziente e il numero di giorni di permanenza in ospedale da apprendere. Alcune osservazioni sono:

Età	CCI	Proc.	Sist.	Diast.	BMI	Giorni
37	2	12	148	106	22.5	4
77	1	7	105	106	38.0	3
83	2	17	82	64	29.2	5

Il modello deve trovare i sei pesi e il bias che minimizzano l'errore tra i giorni stimati e quelli presenti nell'ultima colonna.

1.7.1 Come parte il training

Il training parte da coefficienti iniziali nulli:

$$w_1 = \dots = w_6 = 0, \quad b = 0. \quad (1.23)$$

Con questa scelta, la previsione iniziale per ogni paziente è zero. Il computation server confronta tali previsioni con i giorni di permanenza osservati e usa gli errori per correggere pesi e bias. Il procedimento viene ripetuto epoca dopo epoca: i parametri non sono ottenuti in un singolo passaggio, ma attraverso un miglioramento progressivo del modello.

1.7.2 Quale modello viene recuperato

Il generatore costruisce una durata attesa a partire dalle caratteristiche del paziente. Età, CCI e numero di procedure forniscono un contributo di base; valori pressori o BMI esterni agli intervalli considerati normali aggiungono un ulteriore contributo.

Il dataset non segue quindi una relazione lineare esatta. La regressione apprende una funzione affine che approssima l'andamento complessivo:

$$\widehat{\text{days_in_hospital}} = w^T x + b. \quad (1.24)$$

L'uscita va interpretata come una stima della permanenza e non come un valore clinico certo.

1.7.3 Verifica su un nuovo dato

Una query contiene le sei caratteristiche di un nuovo paziente, per esempio:

$$x_q = (50, 4, 20, 128, 96, 26.7). \quad (1.25)$$

Il modello calcola $\hat{y} = w^T x_q + b$ e restituisce il numero stimato di giorni di permanenza in ospedale. Una volta ottenuti i coefficienti, la query richiede soltanto prodotti e somme e si presta quindi alla valutazione omomorfica.

1.7.4 Lettura dell'esempio nei due scenari

Nello **scenario 1**, il query actor cifra il vettore clinico x_q e lo invia al computation server. Il server applica il modello al dato cifrato e produce

$$\text{Enc}_{pk}(w^T x_q + b), \quad (1.26)$$

cioè la stima cifrata dei giorni di ricovero. Il risultato viene poi decifrato dal SSN.

Nello **scenario 2**, il computation server ha già trasmesso il polinomio cifrato al decryptor. Il SSN custodisce i coefficienti e riceve in chiaro il vettore clinico del paziente, sul quale calcola $w^T x_q + b$. Il valore matematico finale è lo stesso; cambia il punto del sistema in cui i dati del paziente vengono esposti in chiaro.

1.8 Aspetti operativi e limiti

La cifratura omomorfica consente di analizzare dati distribuiti tra strutture sanitarie senza dividerli in forma leggibile. La semplicità della regressione lineare la rende adatta a progetti pilota nei quali la trasparenza del modello è importante.

Restano limiti concreti: CKKS introduce piccole approssimazioni numeriche, il costo computazionale supera quello dell'elaborazione in chiaro e occorre gestire chiavi, audit, responsabilità e validazione clinica.

1.9 Conclusioni

La regressione lineare rappresenta un caso di studio efficace per mostrare come la cifratura omomorfica possa essere applicata a scenari medicali concreti. Il computation server costruisce un polinomio di primo grado, o più in generale una funzione affine multivariata, a partire da dati clinici raccolti dai data provider. Successivamente tale modello può essere interrogato secondo architetture differenti, che riflettono scelte diverse in termini di privacy, fiducia e semplicità operativa.

Lo scenario 1 privilegia la protezione della query e riduce l'esposizione del dato di input al computation server, ma richiede una pipeline più articolata. Lo scenario 2, invece, centralizza la fiducia nel decryptor, semplifica l'interrogazione e può adattarsi bene a una governance sanitaria istituzionale, pur esponendo in chiaro la query al soggetto fiduciario.

Capitolo 2

Protocollo di aggregazione simil prio3 utilizzando tecniche omomorfiche

2.1 Introduzione

Prio3 VDAF è, in modo molto semplice, un protocollo pensato per raccogliere dati da più client e calcolarne un risultato aggregato senza esporre il contributo individuale di ciascuno. L'idea generale è che ogni client non invii direttamente il proprio valore in chiaro a un unico soggetto, ma produca una rappresentazione adatta a essere elaborata da più attori del protocollo. In questo modo, il sistema finale può recuperare il risultato complessivo senza ricostruire facilmente il dato del singolo partecipante.

L'obiettivo di questo capitolo è allora il seguente: replicare in ambiente FHE la forma generale di un protocollo di aggregazione ispirato a Prio3, mantenendo l'idea di fondo di raccolta privata e di validazione dei report, ma semplificando l'architettura. La costruzione che introduciamo, chiamata **fhe-vdaf-0**, usa lo schema omomorfo BGV: i report vengono cifrati dai client, inviati a un unico aggregatore, validati sotto cifratura e poi sommati, in modo che l'attore finale possa apprendere soltanto il risultato aggregato.

La scelta di BGV non è casuale. In questo contesto i report sono valori interi e tutta la logica di validazione viene espressa tramite aritmetica modulare. BGV è quindi naturale perché lavora direttamente su interi modulo p . Questo significa che somme e moltiplicazioni avvengono nello stesso ambiente algebrico in cui vogliamo controllare i report, compreso il comportamento di overflow, che viene assorbito in modo coerente dalla riduzione modulo p .

2.2 Idea generale di fhe-vdaf-0

La struttura di **fhe-vdaf-0** può essere descritta in modo molto semplice:

- (a) il client vuole inviare un valore numerico;
- (b) lo converte in una rappresentazione binaria a slot;
- (c) cifra tale rappresentazione con la chiave pubblica del sistema;

- (d) invia il ciphertext all'aggregatore;
- (e) l'aggregatore valida omomorficamente il report;
- (f) se il report è valido, lo aggiunge alla somma aggregata;
- (g) il collector finale decifra solo il totale.

In pratica, tutti i report cifrati arrivano allo stesso aggregatore. La crittografia di base resta la stessa, ma cambia il modo in cui il sistema distribuisce i compiti e la fiducia tra gli attori.

2.3 Ruoli architetturali e gestione delle chiavi

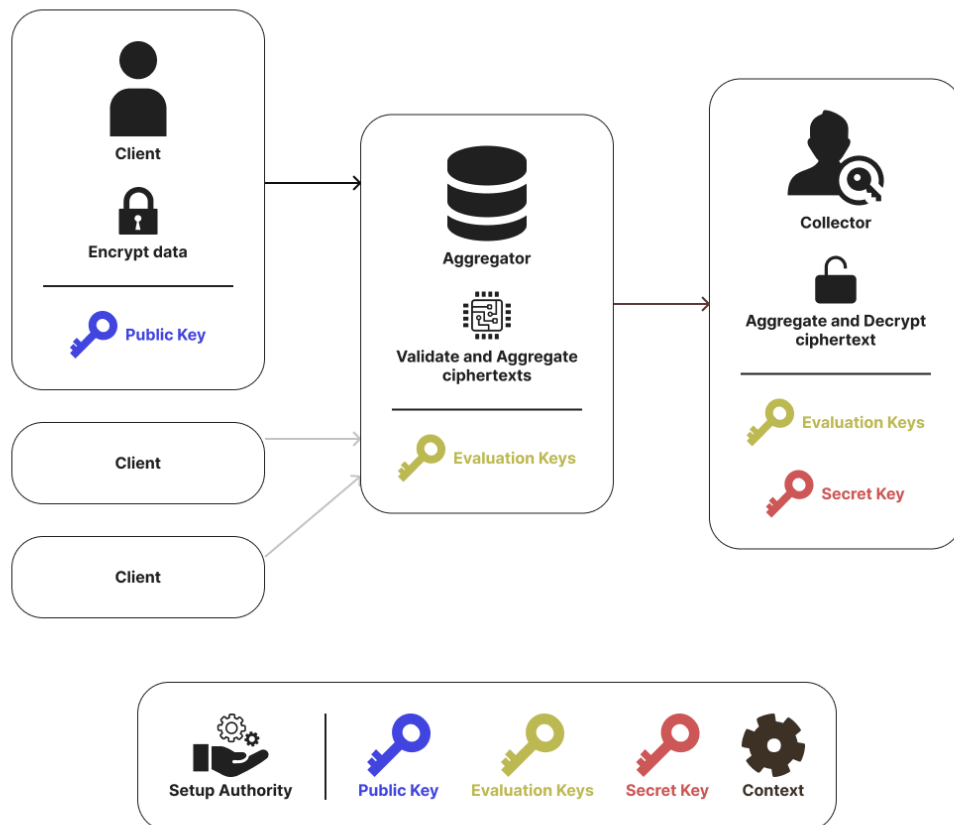


Figura 2.1: Schema di fhe-vdaf-0. Nell'esempio il client invia il numero 19 cifrato.

2.3.1 Attori del sistema

Nel caso di `fhe-vdaf-0` si possono distinguere quattro attori principali:

- il *setup authority*, che crea il contesto crittografico e le chiavi;

- i *client*;
- l'*aggregator*, che riceve i report cifrati;
- il *collector*, che decifra il risultato finale.

2.3.2 Chi possiede quali chiavi

Dal punto di vista della gestione delle chiavi, **fhe-vdaf-0** conserva la distinzione tra cifratura, valutazione omomorfica e decifratura finale.

La distribuzione tipica è la seguente:

- i **client** possiedono il contesto pubblico e la **chiave pubblica** pk , necessaria per cifrare i report;
- l'**aggregator** possiede il contesto crittografico e le **evaluation keys**, cioè le chiavi ausiliarie per eseguire somme, rotazioni e moltiplicazioni omomorfe;
- il **collector** possiede la **chiave segreta** sk e può decifrare il risultato finale;
- il **setup authority** genera inizialmente pk , sk ed evaluation keys, e poi le distribuisce ai ruoli autorizzati.

Questa separazione è importante. L'aggregatore può elaborare i dati cifrati ma, in condizioni normali, non può decifrarli. Il collector può decifrare, ma non riceve i singoli report in chiaro durante la fase di raccolta.

2.3.3 Conseguenze di sicurezza

L'aspetto più delicato di **fhe-vdaf-0** è la concentrazione dello schema presso l'aggregatore. Questo comporta che:

- tutti i ciphertext passano per lo stesso nodo;
- tutta la validazione viene eseguita nello stesso punto;
- non c'è ridondanza fiduciaria tra aggregatori distinti;
- un'eventuale collusione tra aggregatore e soggetto che controlla la chiave segreta comprometterebbe la protezione dei report individuali.

Per questo motivo, **fhe-vdaf-0** è molto semplice da capire, ma meno forte di una soluzione con più aggregatori indipendenti.

2.4 Architettura complessiva in uno scenario di dati aggregati

In **fhe-vdaf-0**, tutti i report vengono cifrati e inviati all'aggregatore. L'aggregatore non deve conoscere il contenuto di ciascun numero, ma deve poter:

- verificare che ogni report sia ben formato;
- scartare i report invalidi;
- sommare i report validi.

Infine, il collector riceve la somma cifrata finale e la decifra, ottenendo solo il totale aggregato.

2.4.1 Schema dei messaggi

Lo schema architetturale può essere rappresentato così, assumendo che il setup authority distribuisca preventivamente le chiavi ai vari attori autorizzati:

$$\text{Client}_i \longrightarrow \text{Aggregator} : \text{Enc}_{pk}(\text{report}_i), \quad (2.1)$$

$$\text{Aggregator} : \text{validate-or-zero}(\text{Enc}_{pk}(\text{report}_i)), \quad (2.2)$$

$$\text{Aggregator} : \sum_i \text{Enc}_{pk}(\text{report}_i), \quad (2.3)$$

$$\text{Aggregator} \longrightarrow \text{Collector} : \text{Enc}_{pk}(\text{totale}), \quad (2.4)$$

$$\text{Collector} : \text{Dec}_{sk}(\text{Enc}_{pk}(\text{totale})). \quad (2.5)$$

2.5 Conversione dell'input in binario

2.5.1 Perché usare una rappresentazione binaria

Il cuore operativo del protocollo è la trasformazione del dato numerico in una sequenza di slot binari. Questa scelta non è un dettaglio marginale: è proprio ciò che consente all'aggregatore di verificare, anche sotto cifratura, che il report abbia una forma lecita.

La scomposizione in bit permette di ridurre il controllo a una proprietà locale:

$$x \in \{0, 1\}. \quad (2.6)$$

2.5.2 Vincolo sul valore massimo

Nel prototipo considerato, il valore massimo consentito è

$$\text{MAX_SIZE} = 100. \quad (2.7)$$

Per rappresentare correttamente tutti i valori interi compresi tra 0 e MAX_SIZE, il numero minimo di bit richiesto si calcola con

$$\text{MAX_BITS} = \lfloor \log_2(\text{MAX_SIZE}) \rfloor + 1, \quad (2.8)$$

valida per MAX_SIZE > 0.

Nel nostro caso:

$$\lfloor \log_2(100) \rfloor + 1 = \lfloor 6.64 \dots \rfloor + 1 = 6 + 1 = 7. \quad (2.9)$$

Quindi

$$\text{MAX_BITS} = 7. \quad (2.10)$$

La codifica non è una rappresentazione binaria standard perfettamente uniforme. I primi sei slot usano i pesi classici

$$1, 2, 4, 8, 16, 32, \quad (2.11)$$

mentre l'ultimo slot usa il peso

$$37. \quad (2.12)$$

La ragione è che si vuole far corrispondere il vettore di tutti uno al valore massimo 100:

$$1 + 2 + 4 + 8 + 16 + 32 + 37 = 100. \quad (2.13)$$

In questo modo, tutti i valori ammessi tra 0 e 100 possono essere rappresentati con sette slot binari.

La logica di codifica seguita dal codice è la seguente. Si considerano prima i primi sei bit, che possono rappresentare direttamente tutti i valori compresi tra 0 e

$$\text{LOW_BITS_MAX_VALUE} = 2^6 - 1 = 63. \quad (2.14)$$

Se il valore da codificare soddisfa

$$v \leq 63, \quad (2.15)$$

allora viene codificato normalmente nei primi sei bit e l'ultimo bit resta a zero.

Se invece

$$v > 63, \quad (2.16)$$

allora il codice:

- (a) pone l'ultimo bit uguale a 1;
- (b) sottrae il suo peso, cioè 37, dal valore originale;
- (c) codifica il valore rimanente nei primi sei bit.

Il valore finale decodificato sarà quindi sempre

$$v = v_{\text{low}} + 37 \cdot b_7. \quad (2.17)$$

2.5.3 Esempi di codifica

Un esempio più interessante è un valore maggiore di 63, ad esempio 64. In questo caso il codice accende l'ultimo bit e poi codifica il resto:

$$64 - 37 = 27. \quad (2.18)$$

Poiché

$$27 = 1 + 2 + 8 + 16, \quad (2.19)$$

si ottiene

$$64 \mapsto [1, 1, 0, 1, 1, 0, 1]. \quad (2.20)$$

Infine, il valore massimo:

$$100 \mapsto [1, 1, 1, 1, 1, 1, 1]. \quad (2.21)$$

Infatti, in questo caso:

$$100 - 37 = 63, \quad (2.22)$$

e 63 viene rappresentato dai primi sei bit tutti a uno:

$$63 = 1 + 2 + 4 + 8 + 16 + 32. \quad (2.23)$$

2.6 Validazione omomorfica dell'input

2.6.1 Controllo che ogni slot sia un bit

Una volta ricevuto un ciphertext, l'aggregatore deve verificare che ogni slot codifichi davvero un bit, cioè un valore 0 oppure 1.

Il controllo fondamentale usa il polinomio elementare

$$f(x) = x(x - 1). \quad (2.24)$$

Quindi:

- se uno slot contiene 0 o 1, il controllo restituisce zero;
- se contiene un altro valore, compare un errore non nullo.

Se il ciphertext contiene gli slot $x_1, \dots, x_{\text{MAX_BITS}}$, l'aggregatore calcola per ogni slot

$$e_i = x_i(x_i - 1) \quad (2.25)$$

e poi somma tutti gli errori:

$$E = \sum_{i=1}^{\text{MAX_BITS}} e_i. \quad (2.26)$$

Se il report è ben formato, allora

$$E = 0. \quad (2.27)$$

Se invece almeno uno slot non è binario, allora

$$E \neq 0. \quad (2.28)$$

2.6.2 Da errore a flag di validità

Il valore E è ancora cifrato. L'aggregatore non può leggerlo direttamente, ma può trasformarlo in un flag di errore sfruttando l'aritmetica del campo finito usato dallo schema BGV.

Nel prototipo viene usato il modulo primo

$$p = 786433. \quad (2.29)$$

Per il piccolo teorema di Fermat, per ogni elemento non nullo vale

$$E^{p-1} = 1 \pmod{p}, \quad (2.30)$$

mentre se $E = 0$ allora

$$E^{p-1} = 0. \quad (2.31)$$

Poiché

$$p - 1 = 786432 = 3 \cdot 2^{18}, \quad (2.32)$$

l'aggregatore può costruire il flag di errore come

$$\text{is_error} = E^{786432}. \quad (2.33)$$

Il significato è:

- $\text{is_error} = 0$ se il report è valido;
- $\text{is_error} = 1$ se il report è invalido.

Da qui si ottiene il flag opposto:

$$\text{is_ok} = 1 - \text{is_error}. \quad (2.34)$$

Quindi:

- $\text{is_ok} = 1$ per i report validi;
- $\text{is_ok} = 0$ per i report invalidi.

2.6.3 Validate-or-zero

A questo punto l'aggregatore moltiplica il report cifrato per il flag di validità:

$$\text{report}_{\text{checked}} = \text{report} \cdot \text{is_ok}. \quad (2.35)$$

Il risultato è molto intuitivo:

- se il report è valido, viene moltiplicato per 1 e resta invariato;
- se il report è invalido, viene moltiplicato per 0 e diventa il vettore nullo.

Questa tecnica è spesso descritta come *validate-or-zero*. In un prototipo dimostrativo è utile perché consente di continuare l'aggregazione senza interrompere l'esecuzione per ogni input scorretto.

2.7 Aggregazione dei report validi

Una volta validati tutti i report, l'aggregatore li somma omomorficamente. Se i report validi sono $c_1, \dots, c_m$, la somma aggregata è

$$C_{\text{sum}} = c_1 + c_2 + \dots + c_m. \quad (2.36)$$

Poiché ogni ciphertext contiene i sette slot della codifica binaria, il risultato della somma è un nuovo vettore cifrato i cui slot rappresentano la somma posizione per posizione dei bit validi ricevuti.

Il collector riceve questo ciphertext finale e lo decifra. Dopo la decifratura, non ottiene direttamente il totale come singolo intero, ma i sette slot aggregati:

$$[s_1, s_2, s_3, s_4, s_5, s_6, s_7]. \quad (2.37)$$

Il totale finale viene quindi ricostruito come

$$\text{totale} = 1 \cdot s_1 + 2 \cdot s_2 + 4 \cdot s_3 + 8 \cdot s_4 + 16 \cdot s_5 + 32 \cdot s_6 + 37 \cdot s_7. \quad (2.38)$$

Questo passaggio è analogo alla funzione di *decode* del valore usata nel progetto.

2.8 Esempio completo su dati aggregati

2.8.1 Codifica dei valori

La codifica binaria a sette slot produce:

$$51 \mapsto [1, 1, 0, 0, 1, 1, 0], \quad (2.39)$$

$$7 \mapsto [1, 1, 1, 0, 0, 0, 0], \quad (2.40)$$

$$4 \mapsto [0, 0, 1, 0, 0, 0, 0], \quad (2.41)$$

$$49 \mapsto [1, 0, 0, 0, 1, 1, 0], \quad (2.42)$$

$$13 \mapsto [1, 0, 1, 1, 0, 0, 0]. \quad (2.43)$$

Ogni vettore viene cifrato con la chiave pubblica pk e inviato all'aggregatore.

2.8.2 Somma per slot

Dopo la decifratura, il collector ottiene la somma per slot:

$$[4, 2, 3, 1, 2, 2, 0]. \quad (2.44)$$

2.8.3 Decodifica finale

Il totale finale si ricostruisce come:

$$\text{totale} = 1 \cdot 4 + 2 \cdot 2 + 4 \cdot 3 + 8 \cdot 1 + 16 \cdot 2 + 32 \cdot 2 + 37 \cdot 0 \quad (2.45)$$

$$= 4 + 4 + 12 + 8 + 32 + 64 + 0 \quad (2.46)$$

$$= 124. \quad (2.47)$$

Il collector apprende dunque solo il totale

$$124, \tag{2.48}$$

senza aver partecipato alla validazione dei singoli report e senza aver ricevuto i report individuali in chiaro durante la fase di raccolta.

2.9 Punti di forza e limiti architetturali

L'architettura adottata presenta una serie di vantaggi pratici, ma anche limiti precisi che vanno esplicitati con chiarezza.

2.9.1 Vantaggi

I vantaggi principali di `fhe-vdaf-0` sono:

- semplicità architetturale;
- minore complessità di orchestrazione;
- minore frammentazione degli artifact e dei flussi;
- facilità di integrazione in sistemi centralizzati.

2.9.2 Limiti

I principali limiti sono:

- assenza di distribuzione fiduciaria tra aggregatori distinti;
- concentrazione di tutti i ciphertext presso lo stesso nodo;
- maggiore esposizione in caso di compromissione dell'aggregatore;
- minore vicinanza alle costruzioni VDAF più robuste.

2.10 Conclusioni

`fhe-vdaf-0` può essere visto come una forma molto semplice di raccolta privata di conteggi basata su cifratura omomorfica e validazione VDAF-like. Il meccanismo centrale è quello della codifica binaria del valore, del controllo elementare di validità e dell'aggregazione cifrata presso un unico nodo.

Il capitolo ha evidenziato tre aspetti principali. Il primo riguarda l'architettura: client, aggregatore unico e collector con chiavi chiaramente separate. Il secondo riguarda la codifica dell'input, che trasforma ogni valore ammesso in un vettore di bit pesati. Il terzo riguarda la validazione, basata su controlli polinomiali elementari ma molto efficaci per riconoscere valori non binari.

In uno scenario reale, questa architettura può essere utile come base sperimentale o come soluzione centralizzata per aggregazioni relativamente semplici. Resta però

importante ricordare che la semplicità di `fhe-vdaf-0` corrisponde anche a una minore distribuzione del trust. Di conseguenza, il suo impiego va valutato nel quadro più ampio della governance del sistema, della gestione delle chiavi e delle garanzie organizzative offerte dall'infrastruttura che lo ospita.